Learning-Augmented Robust Traffic Engineering for Resilient Software-Defined Networks

Final Project Proposal

ECE GY 7363: Communications Networks II - Design and Algorithms

April 2026

Project Focus	Learning-augmented traffic engineering under dynamic demand and failures
Core Methods	Traffic prediction, multi-commodity flow optimization, resilience evaluation
Target Outcome	Lower congestion and stronger performance relative to classical baselines

Abstract

This project proposes a learning-augmented framework for traffic engineering in software-defined networks under time-varying demand and link-failure uncertainty. The main idea is to combine short-horizon traffic prediction with optimization-based multi-commodity flow routing so that the controller can proactively reduce congestion while preserving performance under failures. The machine learning component predicts the next traffic matrix from historical observations, while the optimization component computes routing decisions that minimize maximum link utilization subject to flow conservation and capacity constraints. The resulting framework will be evaluated against shortest-path routing, optimization without prediction, and conservative robust baselines on standard network topologies and synthetic dynamic traffic traces. The expected outcome is a systematic study of when prediction-enhanced traffic engineering improves efficiency and resilience, and when robust non-predictive methods remain preferable.

1. Introduction and Motivation

Communication networks increasingly operate in environments where traffic demand changes rapidly over time and failures can occur with little warning. Traditional shortest-path routing is simple and computationally cheap, but it often reacts poorly to congestion because it does not explicitly reason about capacity sharing across commodities.

Optimization-based traffic engineering improves this situation by accounting for the full demand matrix and link capacities, yet a purely reactive optimization approach may still perform suboptimally when demand shifts between control intervals. At the same time, machine learning has become a practical tool for traffic forecasting, anomaly detection, and resource management, but ML alone does not guarantee feasible routing decisions or provable capacity compliance.

This project is motivated by the observation that prediction and optimization are complementary. If an accurate predictor can estimate the near-future traffic matrix, then a routing optimizer may use that information to allocate capacity more effectively before congestion materializes.

2. Problem Statement

The network is represented by a directed graph G = (V, E), where V is the set of nodes and E is the set of directed links. Each link (i, j) has capacity c_ij, and each source-destination pair is treated as a commodity with time-varying demand d_k(t).

The design problem is to compute routing decisions that satisfy demand while controlling congestion and maintaining strong performance under failures. The central research question is whether short-term traffic prediction improves routing quality and resilience relative to shortest-path and non-predictive optimization baselines in dynamic software-defined networks.

3. Project Objectives

- Formulate a dynamic traffic engineering model for a capacitated communication network.
- Train a machine learning model to predict near-future traffic matrices from historical data.
- Integrate predicted demand into a multi-commodity flow routing optimizer.
- Evaluate resilience under random and critical link-failure scenarios.
- Compare the proposed method against classical and optimization-based baselines.

4. Relevance to Course Themes

The proposal directly addresses network design problem modeling, optimization methods, multi-commodity flow routing, fair network design, resilient design, robust design, and machine-learning-based network management.

It also satisfies the required final-project structure: problem definition, design model, solution methods, and performance evaluation.

5. Technical Approach

At each time step, the controller will observe recent traffic matrices, use a predictive model to estimate the next traffic matrix, solve a routing optimization problem using the predicted demand, and evaluate that routing under the realized traffic and possible link failures.

The end-to-end pipeline is: historical traffic -> prediction model -> routing optimizer -> performance evaluation.

6. Machine Learning Component

The ML module will take a sliding window of recent traffic matrices as input and output a one-step-ahead traffic matrix prediction.

The study will compare simple predictors such as moving average or linear regression against a recurrent model such as LSTM or GRU.

The main model will likely be an LSTM because dynamic traffic exhibits temporal correlation and burst structure that sequence models can capture effectively.

Prediction quality will be measured using MAE, RMSE, and relative traffic prediction error.

7. Optimization Component

The routing problem will be modeled as a multi-commodity flow program. For each commodity k and link (i, j), f_ij^k(t) denotes the amount of traffic routed on that link at time t.

The model will enforce flow conservation, link-capacity constraints, and nonnegativity. To control congestion, an auxiliary utilization variable U(t) will be introduced and minimized so that the sum of flows on each link remains below U(t) times the link capacity.

This yields a minimum-max-utilization routing policy, which is standard, interpretable, and well aligned with traffic engineering objectives.

8. Robustness and Resilience

The project will study resilience in two layers. First, scenario-based evaluation will test nominally optimized routing under random and critical link-failure scenarios.

Second, if time permits, a simplified robust optimization extension will hedge against a predefined set of failures or demand perturbations directly in the solver.

This structure ensures that the project always includes a meaningful resilience component even if the full robust model is computationally expensive.

9. Algorithms and Baselines

- Baseline 1: shortest-path routing without optimization or prediction.
- Baseline 2: multi-commodity flow optimization using only the current observed traffic matrix.
- Baseline 3: conservative non-predictive routing with safety margins or worst-case demand estimates.
- Proposed method: learning-augmented routing that uses the predicted traffic matrix inside the optimization model.

10. Data, Topologies, and Experimental Setup

Experiments will use standard topologies such as NSFNET, Abilene, or GEANT. Additional synthetic topologies may be generated for stress testing.

Traffic matrices will be created from periodic demand trends, random fluctuations, bursty spikes, and hotspot source-destination pairs so the resulting traces are realistic yet reproducible.

Failure scenarios will include no-failure baseline cases, random single-link failures, critical-link failures, and traffic-spike cases combined with failures.

11. Evaluation Metrics

- Maximum link utilization
- Average link utilization
- Fraction of demand satisfied
- Total routing cost
- Average path length
- Number of congested links
- Jain's fairness index
- Performance degradation under failure
- Prediction error of the ML model

12. Expected Contributions

- A complete learning-plus-optimization traffic engineering framework.
- A rigorous comparison of predictive and non-predictive routing methods.
- A resilience analysis under traffic uncertainty and link failures.

- Actionable insight into the settings where ML-guided routing improves performance.

13. Implementation Plan

- Programming language: Python
- Graph handling: networkx
- Optimization: cvxpy or pulp, with Gurobi or CPLEX if available
- Machine learning: PyTorch
- Data processing and plotting: numpy, pandas, matplotlib, seaborn

14. Work Plan and Timeline

- Week 1: finalize problem statement, topology selection, and codebase organization.
- Week 2: build synthetic traffic generation and shortest-path baseline.
- Week 3: implement and validate the multi-commodity flow optimizer.
- Week 4: train baseline and LSTM traffic predictors.
- Week 5: integrate traffic prediction with routing optimization.
- Week 6: run resilience experiments and sensitivity studies.
- Week 7: analyze results and prepare the report, slides, and code package.

15. Risks and Mitigation

Optimization may become slow on larger topologies, so the project will begin with medium-size networks and expand only after the pipeline is validated.

Traffic prediction may not help if the traces are too random, so prediction quality will be evaluated separately from routing performance.

A full robust optimization model may be too expensive to complete, so scenario-based resilience evaluation is the first priority and the robust formulation is treated as an extension.

16. Why This Project is Well Positioned for a High Score

The proposal is highly relevant to the course and combines modern ML ideas with mathematically grounded optimization.

It is deep enough to demonstrate substantial workload but structured enough to remain feasible within the project timeline.

It should produce clear tables, utilization plots, failure-case comparisons, and fairness analyses, which will help the final presentation and report.

17. Conclusion

This proposal presents an end-to-end study of learning-augmented traffic engineering for resilient software-defined networks. By combining short-term traffic prediction with optimization-based multi-commodity flow routing, the project will investigate whether forecast-informed decisions can reduce congestion and improve robustness under failures.

The framework is mathematically grounded, experimentally testable, and directly aligned with the goals of the course, making it a strong candidate for a high-quality final design project.

Planned Software Stack

Component	Planned Tooling
Graph and topology processing	networkx
Numerical processing	numpy, pandas
Optimization	cvxpy or pulp; optional Gurobi/CPLEX
Machine learning	PyTorch
Visualization	matplotlib, seaborn

Table 1. Planned implementation stack for the proposed project.